

Historical Weather Data ICA – Report

A critical evaluation of the software tools, security implications and programming fundamentals used to build the application

Author: CUI JIANGKUN
Student No. : F5743005

1. Introduction

This report accompanies a Python application that processes historical weather data for four cities (Middlesbrough, London, Paris and Toulouse) across two countries. The application is organised into three phases: querying a local SQLite database, generating charts with Matplotlib, and retrieving fresh data from the Open-Meteo archive web-API. The purpose of this report is to critically examine the software tools selected for the assessment and evaluate their appropriateness and limitations, to consider the security and other risk implications of relying on these libraries in real-world settings, and to reflect on how the fundamentals of programming and the standard-library data structures supported the development of the solution. Selected charts produced by the application are embedded as evidence throughout.

2. Software Tools: Appropriateness and Limitations

SQLite (via the `sqlite3` module). SQLite is well suited to this task because the dataset is relational, local and modest in size, and the `sqlite3` module ships with the Python standard library, so no external database server is required (Python Software Foundation, 2024). It supports the joins, grouping, date filtering and aggregate functions that underpin every Phase 1 query, and the same engine drives the Phase 2 chart queries. Its limitations become apparent at scale: SQLite offers only file-level write locking and no native concurrency for many simultaneous writers, so a production weather platform serving concurrent users would require a client-server database such as PostgreSQL together with proper migration control (Owens, 2006). For a single-user analytical workload, however, it is an appropriate and low-friction choice.

Matplotlib. Matplotlib was selected for Phase 2 because it can produce reproducible PNG evidence without a running web dashboard, and it gives fine control over titles, axis labels, legends and figure export (Hunter, 2007). The application uses the non-interactive “Agg” backend so that charts can be rendered and saved head-lessly, which is valuable for automated evidence generation and testing. The main limitation is verbosity: even simple grouped bar charts require manual bar-offset arithmetic and layout tuning, and Matplotlib lacks the interactivity of higher-level libraries such as Plotly or Seaborn. For static, submittable evidence this trade-off is acceptable.

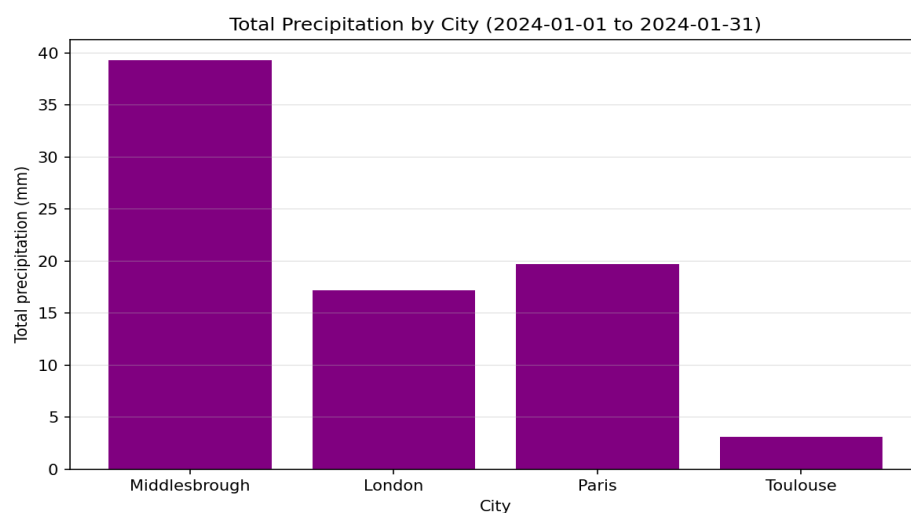
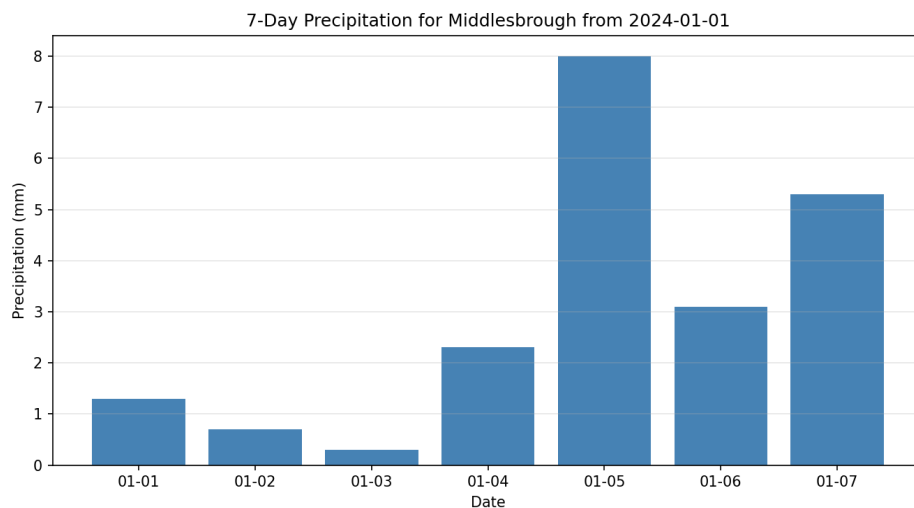
Requests. Phase 3 uses the third-party Requests library to build and send HTTP calls to the Open-Meteo archive API. Requests was chosen over the standard-library `urllib` because it offers a cleaner interface for query parameters, timeouts and JSON decoding, satisfying the brief’s requirement to

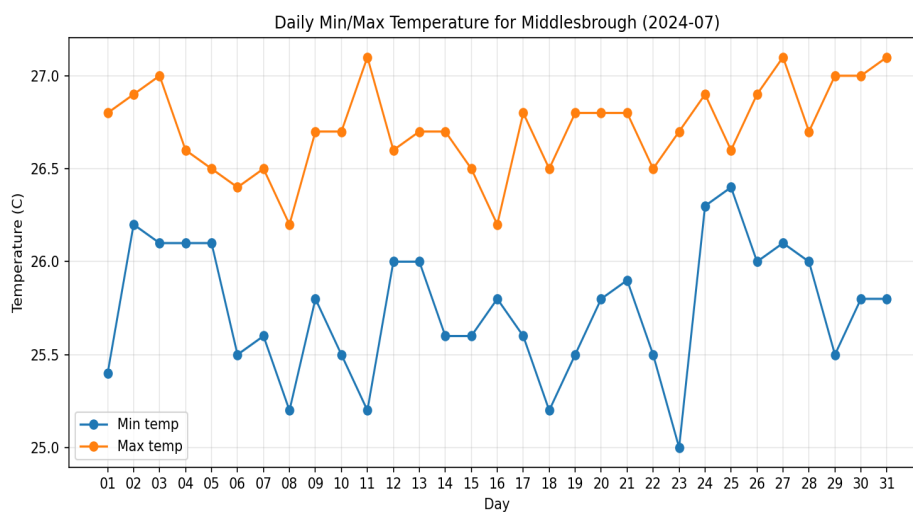
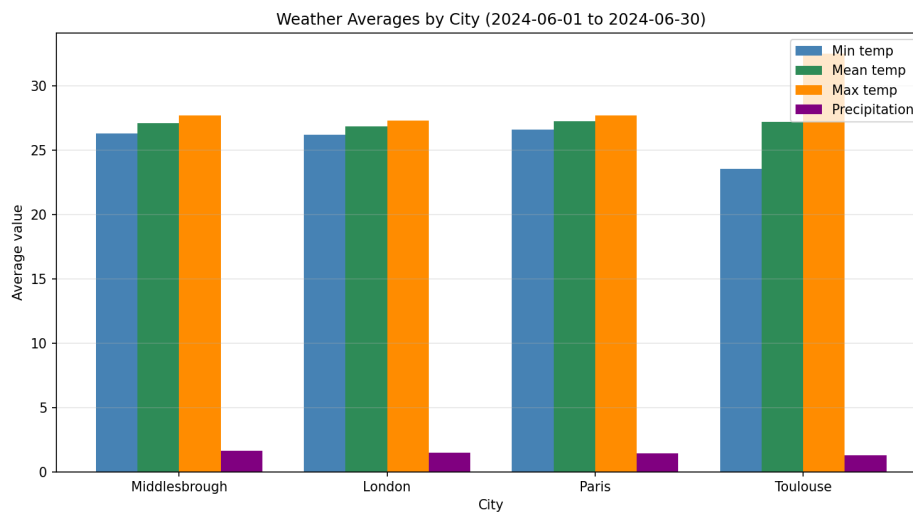
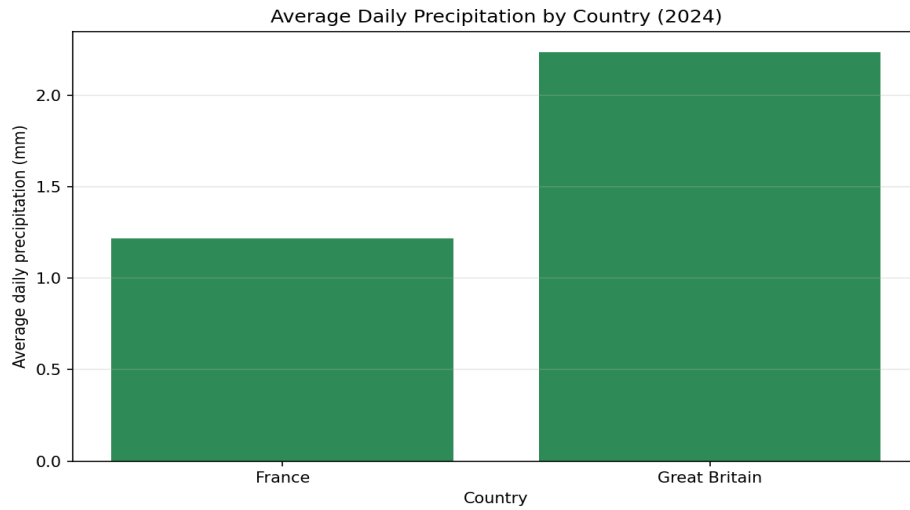
write custom API-handling code rather than relying on the vendor's helper library (Reitz, 2024). Its limitation is that it is a synchronous, blocking client and an external dependency: live calls can fail because of network problems, service changes or malformed responses, all of which must be handled defensively.

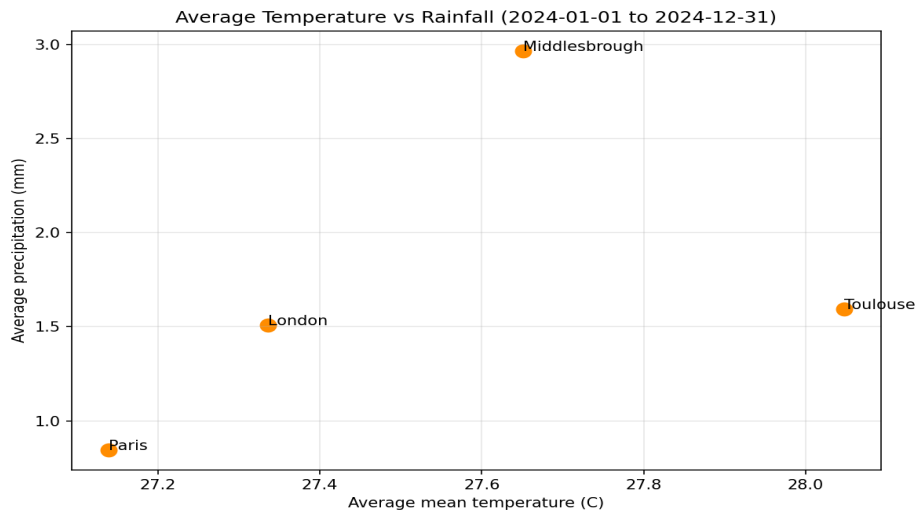
Visual Studio Code. VS Code served as the IDE, providing Python editing, an integrated terminal, source control and SQLite inspection extensions. An editor, however, does not prove correctness; it accelerates development but the project still depends on automated tests and the embedded chart evidence below to demonstrate that the code behaves as intended.

3. Visualisation Evidence

The six charts below are generated directly from the SQLite data by the Phase 2 module and illustrate the analytical value the tools deliver. Beyond the basic charts required by the brief, the queries apply grouping and aggregation to surface comparisons and trends across cities, countries and time.







4. Security and Other Risk Implications

In the context of Data Science and AI applications, the integrity of the data pipeline is as important as model accuracy: a compromised or corrupted database silently degrades every downstream analysis. The most relevant risks here are injection, dependency risk, external-service failure and duplicate or inconsistent data.

SQL injection is the classic risk when user input reaches a database. It is mitigated throughout by parameterised queries, which pass values through placeholders rather than string concatenation – the approach recommended by OWASP as the primary defence against injection (OWASP Foundation, 2021). For example:

```
rows = connection.execute(
    "SELECT ... WHERE daily_weather_entries.city_id = ?",
    (city_id, str(year)),
).fetchall()
```

Input validation reduces both accidental errors and malicious input. Dates and years are validated (including rejecting reversed date ranges) before any query or API call runs, and city lists supplied to chart queries are converted to integers before use. **Dependency risk** is inherent to third-party packages: Requests and Matplotlib expand the attack surface and must be maintained. The project declares its dependencies in requirements.txt, but a production system should also pin exact versions and monitor advisories, since a single compromised transitive package can undermine an entire application (Ohm et al., 2020). **External-service risk** is handled defensively in Phase 3: the response status code is checked, a timeout is set, network exceptions are caught and re-raised with a clear message, and the parser confirms every required field is present before any data is stored. Finally, **duplicate-data risk** is reduced by deleting the existing row for a city/date before inserting refreshed data, so re-running an update does not create duplicates; a production schema would strengthen this further with a unique constraint on (city_id, date).

5. Programming Fundamentals and Data Structures

The standard library's built-in data structures shaped the design directly. **Lists** accumulate query rows and parsed weather records, and their ordered, index-aligned nature is exploited in Phase 3, where the Open-Meteo response arrives as parallel arrays that are zipped together by position:

```

for i in range(len(daily["time"])):
    rows.append({
        "date": daily["time"][i],
        "min_temp": daily["temperature_2m_min"][i],
        "max_temp": daily["temperature_2m_max"][i],
        "mean_temp": daily["temperature_2m_mean"][i],
        "precipitation": daily["precipitation_sum"][i],
    })

```

Dictionaries provide named, self-documenting records. Each parsed day becomes a dictionary keyed by field name, and a single city dictionary carries the id, name, latitude, longitude and timezone required by the API, which is far clearer than threading five positional arguments through several functions. **Functions with single responsibilities** are the core structural idea: query functions return rows, chart functions save PNGs, and Phase 3 splits the work into `get_city`, `download_weather_data`, `parse_daily_weather` and `store_daily_weather`. Because the shared `connect_database` and validation helpers live in one module and are imported by the others, common behaviour is written once and reused, which also makes the code easier to test.

Control flow and exception handling underpin the program's robustness. Conditionals validate ranges, loops iterate over cities and days, and `try/except` blocks convert low-level failures – invalid dates, unknown cities, malformed payloads and network errors – into controlled, meaningful messages. These fundamentals, combined with the `unittest` framework running against temporary copies of the database, allowed the application's behaviour to be verified from the outside without risking the supplied data.

6. Conclusion

The tools selected – SQLite, Matplotlib, Requests and VS Code – were each appropriate for their phase and, within the scope of a single-user analytical application, their limitations were manageable and openly acknowledged. The main real-world risks stem not from the tools themselves but from how they are used: unvalidated input, unmanaged dependencies and unhandled external-service failures. The application addresses these through parameterised queries, input validation, defensive API handling and idempotent database writes. Underpinning all of this, the fundamentals of programming and the standard-library data structures – lists, dictionaries, modular functions and structured exception handling – provided a simple but reliable foundation on which the three phases were built.

References

- Hunter, J.D. (2007) 'Matplotlib: a 2D graphics environment', *Computing in Science & Engineering*, 9(3), pp. 90–95.
- Ohm, M., Plate, H., Sykosch, A. and Meier, M. (2020) 'Backstabber's knife collection: a review of open source software supply chain attacks', in *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2020)*. Cham: Springer, pp. 23–43.
- Open-Meteo (2024) *Historical Weather API documentation*. Available at: <https://open-meteo.com/en/docs/historical-weather-api> (Accessed: 12 July 2026).
- OWASP Foundation (2021) *SQL injection prevention cheat sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html (Accessed: 12 July 2026).
- Owens, M. (2006) *The definitive guide to SQLite*. Berkeley, CA: Apress.

Python Software Foundation (2024) *sqlite3 — DB-API 2.0 interface for SQLite databases*. Available at: <https://docs.python.org/3/library/sqlite3.html> (Accessed: 12 July 2026).

Reitz, K. (2024) *Requests: HTTP for Humans documentation*. Available at: <https://requests.readthedocs.io/> (Accessed: 12 July 2026).